

## 第9章 拷贝构造和赋值函数

### 9.1 拷贝构造函数

拷贝构造函数是一类特殊的成员函数，也是特殊的构造函数，因此构造函数中的一般规则同样适用于拷贝构造函数。

#### 9.1.1 拷贝的必要

从名字可以看出，拷贝构造函数用于：根据一个已存在的本类对象，构造出一个新对象，这个新对象应该是样本对象的“复制品”。这种复制的过程在程序设计中是必要的。例如：

```
1  class Book {
2  public:
3      //合并两本书
4      void join( Book other ) {
5          mNum += other.mNum;
6      }
7      //修订得到新书
8      Book revise( ) {
9          Book newBook;
10         newBook.mPages = mPages + 20;
11         return newBook;
12     }
13     // getter
14     int pageCount( ) const { return mPages;}
15 private:
16     int mPages = 100;
17 };
18 int main( ) {
19     Book book1;
20     Book book2;
21     Book book3(book1);    //等价于 Book book3 = book1;
22
23     book1.join(book2);
24     int total = book1.revise( ).pageCount( );
25 }
```

在语句 `Book book3(book1);` 执行之前，`book3` 并不存在，因此这行代码明确表示要构造一个全新的对象 `book3`，并且要求它与 `book1` 具有相同的属性和状态，这就是拷贝构造的含义。

当执行 `book1.join(book2);` 时，`book2` 作为实参传递给 `join` 函数。由于 `join` 函数的参数 `other`

是按值传递的，这意味着需要在栈上创建一个新的 `Book` 对象 `other`，它是以 `book2` 为样本构建的，这一过程同样依赖于拷贝构造函数来完成。

此外，`book1.revise()` 函数返回的是一个在栈区内创建的 `'newBook'` 对象。然而，当 `'revise'` 函数结束时，`'newBook'` 将被销毁。为了确保返回的对象仍然有效且内容与 `'newBook'` 相同，必须通过拷贝构造函数在函数外部创建一个新的 `'Book'` 对象。

### 9.1.2 拷贝构造函数的原型

拷贝构造函数的访问控制可以是 `public`、`protected` 或 `private`，但其格式为：

---

格式 1: `T(const T &);`

格式 2: `T( T & );`

---

拷贝构造函数仅有一个显式的参数，用于指定要复制的对象样本，且该参数必须是引用类型，不能是指针或对象型。首先，指针类型的参数无法满足拷贝构造的语义要求；其次，如果使用对象类型作为参数，在调用拷贝构造函数时，参数传递会触发再次调用拷贝构造函数以便在栈上创建临时对象，从而导致递归调用，最终由于无限递归引发编译错误。

对于形参是否使用 `const` 修饰，应根据拷贝构造函数的具体语义需求来决定。如果不需要修改传入的样本对象，则应选择带有 `const` 的形式；而如果需要修改传入的样本对象，则不应使用 `const`。例如：

```

1  class Owner1 {
2  public:
3      //其它略
4      Owner1( const Owner1 & oldOwner) {
5          mpInt = new Owner1( *oldOwner.mpInt );
6      }
7  private:
8      int * mpInt;
9  };
10 class Owner2 {
11 public:
12     //其它略
13     Owner2( Owner2 & oldOwner) {
14         mpInt = oldOwner.mpInt;
15         oldOwner.mpInt = nullptr;
16     }
17 private:
18     int * mpInt;

```

```
19 };
```

在 Owner1 中，拷贝构造函数执行的是常规的拷贝操作，实参对象无需被修改，因此形参类型可以是带 const 的引用。而在 Owner2 中，拷贝构造函数实现了所有权转移的语义：当拷贝一个对象时，将原对象对 mpInt 指向的整数的所有权转移给新对象，这要求在拷贝过程中必须能够修改原对象，因此形参类型应该是非 const 的引用(即 Owner&)。

### 9.1.3 缺省拷贝构造函数

如果未显式定义拷贝构造函数，编译器将提供一个缺省的公有拷贝构造函数，也称默认拷贝构造函数。其格式为：

---

格式: `T( const T & );`

---

在拷贝过程中，需要将样本对象的数据成员复制到新对象中。对于数据成员的复制策略，缺省的拷贝构造函数执行的是“浅拷贝”，具体含义如下：

- 对于内置类型、指针和引用类型的数据成员，直接复制其二进制内容。这意味着不会区分数据的具体类型和大小，而是通过高效的内存块复制方式处理，甚至可以一次性复制多个连续的数据成员，这种拷贝方式也称为位拷贝。
- 对于对象类型的数据成员，将调用该对象类型的拷贝构造函数进行复制。这些拷贝构造函数可以是用户自定义的，也可以是编译器提供的缺省版本。

例如：

```
1  #include <iostream>
2  using namespace std;
3  struct B {
4      B( ) = default;
5      B( const B & rhs ) { mVal = rhs.mVal + mVal*2; }
6      int mVal = 10;
7  };
8  struct A {
9      A( ) = default;
10     int mA = 80;
11     int * mpInt = nullptr;
12     B mB;
13 };
14 int main( ) {
15     int aInt = 300;
16     A a1;
```

```

17     a1.mpInt = &aInt;
18
19     A a2( a1 );
20     cout << "a1.mA = " << a1.mA << endl;           //输出 80
21     cout << "a1.mB.mVal = " << a1.mB.mVal << endl; //输出 10
22     cout << "*a1.mpInt = " << *a1.mpInt << endl;   //输出 300
23
24     cout << "a2.mA = " << a2.mA << endl;           //输出 80
25     cout << "a2.mB.mVal = " << a2.mB.mVal << endl; //输出 30
26     cout << "*a2.mpInt = " << *a2.mpInt << endl;   //输出 300
27 }

```

在拷贝构造 a2 时，类 A 的缺省拷贝构造函数会直接按二进制块拷贝 a1.mA 和 a1.mpInt 的值。对于成员变量 a1.mB，则会调用类 B 的拷贝构造函数进行拷贝。如果类 B 中自定义了拷贝构造函数，则使用自定义的；否则，将使用类 B 的缺省拷贝构造函数。

#### 9.1.4 "浅拷贝"的不足

缺省拷贝构造函数采用“浅拷贝”策略，有时无法满足需求，特别是在类中包含指针成员时。例如：

```

1  class B {
2  };
3  class A {
4  public:
5      A( ) :mpB( new B ) { }
6      ~A( ) { delete mpB; }
7  private:
8      B * mpB;
9  };
10 int main( ) {
11     A a1;
12     A a2( a1 );
13 }

```

A 类使用了缺省的拷贝构造函数，这会导致“浅拷贝”。当释放 a1 和 a2 时，堆内存会被重复释放，从而引发访问非法地址的运行时错误。为了解决这个问题，需要自定义拷贝构造函数以实现“深拷贝”。

#### 9.1.5 自定义拷贝构造函数

相对于缺省拷贝构造函数的“浅拷贝”，通过自定义拷贝构造函数可以实现“深拷贝”。这里的“深

拷贝”，既可以是物理意义上的“深拷贝”，也可以是逻辑意义上的“深拷贝”。由于是用户自定义的，编译器会假定该自定义拷贝构造函数已经满足了用户的期望和需求，因此不会进一步检查具体的实现细节。即使自定义的拷贝构造函数并未执行任何实际的拷贝操作，编译器也会认为这是有效的“深拷贝”。通过自定义拷贝构造函数，我们可以重新定义上述示例：

```

1  class B { };
2  class A {
3  public:
4      A( ) :mpB( new B ) { }
5      ~A( ) { delete mpB; }
6      //自定义拷贝构造函数
7      A(const A & rhs ) {
8          mpB = new B( *rhs.mpB );
9      }
10 private:
11     B * mpB;
12 };
13 int main( ) {
14     A a1;
15     A a2( a1 );
16 }

```

这里的自定义拷贝构造函数，实现的就是物理意义上的“深拷贝”。

自定义拷贝构造函数时，以下三种方式均可：

格式 1: `T( [const] T & rhs );`

格式 2: `T( [const] T & rhs ) = default;`

格式 3: `T( [const] T & rhs ) = delete;`

格式 1 是指常规意义上的自定义拷贝构造函数；格式 2 相当于使用缺省的拷贝构造函数；格式 3 则是禁用拷贝构造函数。需要特别强调的是，一旦自定义了拷贝构造函数，编译器将不再提供缺省的无参构造函数。这是因为编译器视拷贝构造函数为一种特殊的构造函数，因此自定义拷贝构造函数也被视为自定义了构造函数。

以下是需要自定义拷贝构造函数的几种常见情况：

1. 实现物理意义上的“深拷贝”。这是最常见的自定义拷贝构造函数的原因。
2. 禁止拷贝和传值。

在 C++11 之前，可以通过多种方式禁止拷贝和传值。下面给出 3 种不同的实现方式：

```

1  //定义方式 1

```

```

2  class My {
3  private://私有的, 自定义, 有实现
4      My(const My & rhs) { /* 略 */}
5  private:
6      void f( );
7  };
8
9  //定义方式 2
10 class My {
11 public: //公有的, 自定义, 但无实现
12     My(const My & rhs);
13 private:
14     void f( );
15 };
16
17 //定义方式 3
18 class My {
19 private: //私有的, 自定义, 但无实现
20     My(const My & rhs);
21 private:
22     void f( );
23 };

```

但对于下面相同的代码, 却有着不同的编译结果:

```

1  inline void My::f( ) {
2      My m1;
3      //以下语句, 方式 1 时合法, 方式 2 时非法且链接错误, 方式 3 时非法且链接错误
4      My m2(m1);
5  }
6  int main( ) {
7      My m1;
8      //以下语句, 方式 1 时编译错误, 方式 2 时链接错误, 方式 3 时编译错误
9      My m2(m1); //非法
10 }

```

从对比结果可见, 第 3 种方式禁止了所有拷贝构造的可能, 而且做到了在编译期间尽早地给出错误报告, 也是 C++11 以前推荐的方式。

在 C++11 之后, 禁止拷贝和传值的写法要更规范和简洁。例如:

```

1  class My {
2  public:
3      My(const My & rhs) = delete; //公有或私有都可以
4  private:

```

```

5     void f( ) {
6         My m1;
7         My m2(m1); //编译错误
8     }
9 };
10 int main( ) {
11     My m1;
12     My m2(m1); //编译错误
13 }

```

3. 实现具有特殊目的的“深拷贝”，也就是逻辑意义上的“深拷贝”。例如计数、所有权转移或实现单例模式等，这些自定义拷贝构造函数的实现逻辑完全取决于需求。例如 9.1.2 一节中 Owner2 类的拷贝构造函数实现：

```

12     //其它略
13     Owner2( Owner2 & oldOwner) {
14         mpInt = oldOwner.mpInt;
15         oldOwner.mpInt = nullptr;
16     }
17     //其它略

```

在对象的拷贝过程中，实现了整数所有权从样本对象到新对象的转移，而不是简单的指针复制，这就属于逻辑上的“深拷贝”。

### 9.1.6 转移构造函数

在新标准下，还增加了一类拷贝构造函数，称为转移构造函数，其在类 T 中的常用格式为：

---

格式 1:  $T( [ \text{类型修饰} ] T \&\& aT );$

格式 2:  $T( [ \text{类型修饰} ] T \&\& aT ) \text{noexcept};$

---

引入转移构造函数的目的是优化拷贝构造的效率。由于涉及到转移语义，这里只简单解释一下。通常情况下，拷贝构造函数会确保样本对象在拷贝后仍然存在，然而，如果明确知道样本对象不再需要保留，那么我们可以通过转移构造函数进行更高效的拷贝和资源管理。举例来说，当一个样本对象拥有系统资源(如堆内存)的所有权时，传统的拷贝构造需要执行“深拷贝”，这可能会带来较大的开销。而转移构造函数可以在已知样本对象不需要保留的情况下，直接将资源的所有权转移给新对象，从而提高程序的执行效率。转移构造函数就是针对这种情况的，当样本对象是右值或将亡值时，编译器会优先使用转移构造函数，而不是普通的拷贝构造函数，从而提高程序的执行效率。这里的右值可以是普通右值或者通过 `std::move` 函数转换为右值的左值，而将亡值通常是临时量或匿

名对象，如进行返回值优化(RVO)时返回的匿名对象就是一个将亡值。例如：

```

1  #include <iostream>
2  using namespace std;
3  class A {
4  public:
5      A( int n ) :mpNum( new int( n ) ) {
6          cout << "A(int)" << endl;
7      }
8      ~A( ) { delete mpNum; }
9      //普通拷贝构造
10     A( const A & rhs ) :mpNum( new int( *rhs.mpNum ) ) {
11         cout << "A(const A &)" << endl;
12     }
13     //转移构造
14     A( A && rhs ) noexcept :mpNum( rhs.mpNum ) {
15         rhs.mpNum = nullptr;    //注意
16         cout << "A( A && )" << endl;
17     }
18     int num( ) const { return mpNum ? *mpNum : 0; }
19     int * mpNum = nullptr;
20 };
21 A f( A a ) {
22     cout << "Call f( A a)" << endl;
23     return a;
24 }
25 int main( ) {
26     A a1( 1 ); //输出 A(int)
27     //输出 *a1.mpNum = 1
28     cout << "*a1.mpNum = " << a1.num( ) << endl << endl;
29     A a2( a1 ); //输出 A(const A &)
30     //输出 *a2.mpNum = 1
31     cout << "*a2.mpNum = " << a2.num( ) << endl << endl;
32     A a3 = f( A( 3 ) ); //输出 A(int)、Call f( A a)、A( A && )
33     //输出 *a3.mpNum = 3
34     cout << "*a3.mpNum = " << a3.num( ) << endl << endl;
35     A a4 = f( a3 ); //输出 A(const A &)、Call f( A a)、A( A && )
36     //输出 *a3.mpNum = 3 、 *a4.mpNum = 3
37     cout << "*a3.mpNum = " << a3.num( ) << endl;
38     cout << "*a4.mpNum = " << a4.num( ) << endl << endl;
39     A a5( std::move( a4 ) ); //输出 A( A && )
40     //输出 *a4.mpNum = 0 、 *a5.mpNum = 3
41     cout << "*a4.mpNum = " << a4.num( ) << endl;
42     cout << "*a5.mpNum = " << a5.num( ) << endl;

```



```
43 }
```

注意其中的三条语句 `A a3 = f( A(3) );`、`A a4 = f( a3 );`和 `A a5( std::move( a4 ) );`在执行过程中的差异。

## 9.2 赋值函数

赋值和拷贝的语义不同：对象拷贝是指从无到有地创建一个全新的对象，而对象赋值则是指修改一个已存在的对象。在类中，我们可以通过定义赋值函数来指定对象间赋值的语义。

### 9.2.1 缺省的赋值函数

如果类中没有显式定义赋值函数，编译器会提供一个缺省的赋值函数，其效果等同于显式地定义公有成员：

---

格式： `T & operator=(const T &rhs) = default;`

---

与缺省拷贝构造函数中的“浅拷贝”类似，缺省赋值函数执行的是“浅赋值”：对于内置类型、指针和引用，它进行位赋值；对于类类型的对象成员，则调用相应类型的赋值函数进行赋值。

如同“浅拷贝”存在局限性，缺省赋值函数的“浅赋值”同样存在不足。例如：

```
1  class B {
2  };
3  class A {
4  public:
5      A( ) :mpB( new B ) { }
6      ~A( ) { delete mpB; }
7  private:
8      B * mpB;
9  };
10 int main( ) {
11     A a1;
12     A a2;
13     a2 = a1;
14 }
```

由于类 A 和类 B 没有自定义赋值函数，因此它们使用编译器提供的缺省赋值函数。执行语句 `a2=a1;` 后，`a1` 和 `a2` 中的 `mpB` 指向了相同的 B 类对象。当 `main` 函数结束时，`a2` 和 `a1` 会自动析构。此时，`a2` 可以正常释放其资源，但在释放 `a1` 时，`mpB` 指向的对象已经在释放 `a2` 对象的过程中释放过了，这会导致重复释放问题，从而引发运行时错误。

### 9.2.2 自定义赋值函数的实现

当缺省的赋值函数无法满足用户需求时，可以通过自定义赋值函数来指定所需的赋值语义。例如，对于之前例子中的问题，可以自定义赋值函数：

```

1  class B {
2  };
3  class A {
4  public:
5      A( ) :mpB( new B ) { }
6      ~A( ) { delete mpB; }
7      //拷贝构造函数
8      A(const A & rhs) : mpB( new B( *rhs.mpB)) { }
9      //自定义赋值函数
10     A & operator=(const A & rhs) {
11         if( this != &rhs ) {
12             delete mpB;
13             mpB = new B( *rhs.mpB );
14         }
15         return *this;
16     }
17 private:
18     B * mpB;
19 };
20 int main( ) {
21     A a1;
22     A a2;
23     a2 = a1;
24 }
```

通过自定义赋值函数和拷贝构造函数，实现了类 A 对象的“深赋值”和“深拷贝”。这样，释放对象时就不会产生重复释放的问题了。

关于自定义赋值函数，还有以下几点说明：

1. 赋值函数通常按引用返回。自定义赋值函数实际上是通过重载赋值运算符实现自定义类型间的赋值操作，重载赋值运算符时，应保持用户使用运算符的习惯，允许类似 `a=b=c` 和 `(a=b)=c` 的链式赋值。为此，赋值函数应返回引用。
2. 赋值函数不能声明为 `const` 成员函数，因为赋值操作需要修改对象的状态。
3. 赋值函数的参数类型没有严格限制。它可以接受自定义类型或内置类型，可以是引用或值传递，也可以是常量或非常量。例如，下面的赋值函数也是可以的。

```

1  class T {
2  public:
3      T & operator=( int );
4      T & operator=( const B & b );
5      T & operator=( T t);
6  };

```

虽然在语法上没有参数类型的限制，但在实践中，强烈建议只定义本类类型为参数的赋值函数，如：

格式1: `T & operator=(const T&);`

格式2: `T & operator=( T & );`

格式3: `T & operator=( T );`

这样可以禁止不同类型对象间的赋值操作，有效避免通过隐式类型转换而引入一些意外的赋值行为。

4. 在实现赋值函数时，要考虑自赋值的情况。如上边的自定义赋值函数：

```

1  class A {
2      //自定义赋值函数
3      A & operator=(const A & rhs) {
4          if( this != & rhs) {
5              delete mpB;
6              mpB = new B( *rhs.mpB );
7          }
8          return *this;
9      }
10     /* 其它略 */
11 };

```

如果没有自赋值检查，可能会导致内存非法使用的问题，例如，`a = a;`的情况。此外，上述实现代码既不是线程安全的，也不是异常安全的。例如，如果 `new B(*rhs.mpB)` 过程中抛出异常，函数将被终止，导致原对象变为无效状态。因此，在实现中更推荐使用如 `swap` 或 `exchange` 等安全的库函数，改进后的代码如下：

```

1  class A {
2      //自定义赋值函数
3      A & operator=(const A & rhs) {
4          A temp(rhs);
5          swap(mpB,temp.mpB);
6          return *this;
7      }

```

```

8      //其它略
9  };
10 或
11  class A {
12      //自定义赋值函数
13      A & operator=( A rhs) {
14          swap(mpB, rhs.mpB);
15          return *this;
16      }
17      //其它略
18  };

```

swap 函数总是安全的。通过 swap 操作，局部对象获得了原对象中的 mpB，而原对象则获取了局部对象中的 mpB。当函数结束时，局部对象会被自动释放，其所拥有的 mpB 也会随之自动释放，从而完美实现赋值功能。在此过程中，如果在调用 swap 之前发生异常(例如拷贝构造局部对象失败)，函数将立即终止，并在终止前释放所有局部对象，但原对象不会受到任何影响，仍然是一个有效的 A 类对象，这体现了异常的安全性。

5. 自定义赋值函数和拷贝构造函数应当同时实现。对于这两者，应该遵循这样的原则：要么都使用缺省版本，要么都自定义实现。

### 9.2.3 转移赋值函数

类似于转移构造函数，当实参是一个右值或将亡值时，也可以优化赋值函数，这种函数就是转移赋值函数。例如，给上边的类 A 增加一个转移赋值函数：

```

1  struct B {};
2  class A {
3  public:
4      A( ) : mpB( new B ) { }
5      ~A( ) { delete mpB; }
6      //拷贝构造函数
7      A(const A & rhs) : mpB( new B( *rhs.mpB)) { }
8      //自定义赋值函数
9      A & operator=( const A & rhs) {
10         A temp(rhs);
11         swap(mpB, temp.mpB);
12         return *this;
13     }
14     //自定义转移赋值函数
15     A & operator=( A && rhs) noexcept {
16         swap(mpB, rhs.mpB);

```

```
17         return *this;
18     }
19 private:
20     B * mpB;
21 };
22 A f( ) { return A( ); }
23 int main( ) {
24     A a1;
25     A a2;
26     A a3( a2 );
27     a2 = a3;    //调用 A & operator=( const A & rhs)
28     a1 = f( );  //调用 A & operator=( A && rhs) noexcept
29     a3 = A( );  //调用 A & operator=( A && rhs) noexcept
30 }
```